

Kinematic Axis of the Human State Interface: Live Implementation

Bemnet Mussa*
Synheart AI

June 27, 2026

Abstract

Five deterministic kinematic axes run live, on-device, from a single phone sensor stream. Each axis maps raw motion and field data to a reading about the wearer’s physical state: activity, movement regularity, posture, locomotion, and an overall summary. This report covers the live implementation and the design approach behind it: how one shared sensor stream is acquired, buffered, resampled, and fanned out to the five axes; how a load-bearing startup calibration establishes the references the axes depend on; and how the system degrades gracefully when a sensor is missing. The per-axis mathematics, derivations, and validation are not restated here; each axis points to its own notebook, which remains the authoritative source for its method and results.

1 Introduction

The Human State Interface (HSI) is a standardized, privacy-first, on-device contract for inferred human state. Rather than exporting raw sensor traces, an HSI source emits a structured JSON reading: a small, named set of state estimates that downstream consumers can rely on without ever handling the underlying signal. The contract is the product; the sensing pipeline behind it is an implementation detail that each source is free to choose, provided the output stays within the agreed schema.

The *kinematic axis* of the HSI is one such source. It turns a phone’s inertial and magnetic sensor data into human-state readings. It is built around three design principles:

- **On-device.** All computation happens where the data is produced. No sensor stream leaves the phone; only the resulting state readings do.
- **Deterministic.** Every reading is produced by an explicit rule over named features, not by a learned model. The same window of data always yields the same reading, and the reason for a reading can be read directly off the rule and its thresholds.
- **Privacy-first.** The readings are abstractions of state, not reconstructions of activity. Keeping the inference on-device and rule-based means there is no raw-data egress and no opaque model to audit.

*Source code and per-axis notebooks: <https://github.com/BemnetMussa/Kinematics-axis>

Why derived rules rather than hand-tuning or a learned model. The deterministic principle is a deliberate choice, and the rules themselves are *derived* rather than picked by eye. A learned classifier would need labelled data on the device, would drift across phones and body placements, and would be opaque to audit — all at odds with an on-device, privacy-first contract. Thresholds hand-tuned by inspection would be just as fragile and far harder to defend. Instead each axis is built on an explicit physical quantity — an energy, a cadence, an autocorrelation, an angle, a field deviation — and, where one exists, an established measurement method from the literature. The detailed derivation and validation for each axis is carried out in its notebook; that is where the mathematical work lives, and it is what lets every reading trace back to a physical cause and a documented threshold rather than be asserted. This report describes how those derived rules are wired together and run live.

The kinematic axis is implemented as five readings, each a small axis of the human state, computed from the same stream:

- **activity_state** — how hard the body is working (sedentary / walking / running / cycling).
- **movement_regularity** — how smooth and repeating the motion is, as a continuous score in $[0, 1]$.
- **postural_state** — which way is down relative to the body (standing / sitting / in_motion / lying / device_resting).
- **locomotion_state** — whether the wearer is moving through space (stationary / on_foot / in_vehicle).
- **overall_state** — the four combined into a single situation label, an exertion level, and a heart-rate context note.

The remainder of this report describes how these five run together as one live system.

2 System Architecture

The live system is a single loop over one shared sensor stream. The same window of data drives all five axes every iteration; nothing is computed twice and no axis owns its own acquisition.

One shared stream. Sensor data arrives from a phone running Phyphox, exposed over WiFi and polled by the host [7]. The host reads the experiment’s buffer mapping directly from the device configuration — accelerometer (raw, with gravity), linear acceleration, gyroscope, magnetometer, and gravity — rather than guessing channel names. Incoming samples (nominally ~ 50 Hz) accumulate in a per-sensor buffer that retains a few seconds of history. On each iteration the buffer is resampled onto a common 50 Hz grid over the requested window, so every axis sees the same time base regardless of the device’s native per-sensor rates.

Fan-out to five axes. The resampled window is handed to the five axes in turn. Two window lengths are used: a short 2.56 s window (128 samples) for **activity_state**, and a 5 s window (250 samples) for **movement_regularity**, **postural_state**, and **locomotion_state**. **overall_state** consumes the four readings rather than the window itself. No axis re-acquires or re-buffers data; acquisition

is shared and the axes are pure functions of their window (and, where needed, the calibration references of Section 3).

Two views. The same set of readings is presented two ways. A terminal dashboard gives the engineering view: per-axis reading, a confidence figure, and the deciding evidence (the feature values that drove the rule), printed each window. A human-facing web view, served locally from a background thread, reflects exactly those numbers as a plain sentence with a confidence word, per-axis cards, a posture tilt dial, an exertion meter, and a live sensor strip. The web view performs no classification of its own; it is a presentation of the readings the pipeline already produced.

Modular layout. The implementation is split so that all decision logic is isolated from acquisition and presentation:

Module	Responsibility
<code>axes/</code>	the five axis rules, one file each
<code>core/config.py</code>	every threshold and constant, in one place
<code>core/features.py</code>	shared feature math: body acceleration, magnitude, autocorrelation, angles, field
<code>interface/sources.py</code>	the swappable sensor source behind one <code>get_sensor_window</code> contract
<code>interface/dashboard.py</code>	startup calibration and the terminal readings view
<code>interface/web.py</code>	the human-facing web view

This layout is a refactor of an earlier single-file implementation. The split preserved behaviour exactly: it was verified byte-identical (md5) against the original single-file implementation’s output, so the modular structure carries no logic change.

Graceful degradation. Acquisition is swappable behind a single `get_sensor_window` call, with a live Phyphox source and an offline synthetic source sharing one contract. A sensor that is not exposed by the experiment, or that stalls, simply stays absent in the window. Each axis checks for the inputs it needs and, if they are missing, returns a null reading with a printed note rather than raising. The loop additionally wraps each axis call so that an exception in one axis is reported and skipped instead of killing the stream. A missing magnetometer, for example, disables the stationary-versus-vehicle branch of `locomotion_state` but leaves the other four axes running.

3 Calibration

Calibration is treated as its own concern because the axes depend on it. Two of the references the rules use — the postural “up” direction and the magnetometer baseline — are not properties of the phone but of how the phone is worn and where the wearer is standing. They must be captured at startup, in the deployment position, before any reading can be trusted.

Startup procedure. The wearer places the phone in its deployment position (the front trouser pocket), stands up straight, and holds still through a short get-ready countdown. After the countdown, a capture window records the standing configuration. The mean accelerometer vector over

that window is the standing gravity direction; it becomes the postural “up” reference. The magnetometer over the same window establishes the field baseline (magnitude and inclination) that `locomotion_state` measures deviations against.

Buffer flush. On a live source the buffer is flushed once before the countdown begins. Without this, the capture would draw on the backlog accumulated between connecting and starting — stale, pre-position data. Flushing leaves the source’s clock at the phone’s current time, so the countdown measures real elapsed time and the captured window is live data from the standing position, not replay.

Validation and confirmation. A bad baseline silently corrupts every later reading, so the capture is checked before it is accepted, and a PASS/FAIL verdict is printed:

- **Gravity $\approx 1g$.** The magnitude of the captured “up” vector must sit near $1g$. A value far from $1g$ means the phone was accelerating — i.e. moving — during capture, so the direction is not a clean gravity reference.
- **Held still.** The per-sample acceleration-magnitude spread over the window must be small. A large spread means the wearer moved during capture.
- **Field sane.** The magnetic field magnitude must fall in a plausible geomagnetic range ($\sim 20\text{--}70\ \mu\text{T}$). A value outside this range indicates local interference (steel, a magnet, a motor) rather than the Earth’s field, which would poison the locomotion baseline.

If any check fails the verdict is reported as suspect, with the failing quantity shown, so the run can be restarted before a corrupted baseline is used.

Geometry. Figure 1 shows the placement the calibration assumes. With the phone in the front trouser pocket and the wearer standing, gravity points down the long axis of the thigh at $\sim 1g$, and the captured “up” reference is the opposite direction. The postural tilt used at run time is the angle θ between the current mean gravity vector and this calibrated standing “up”: $\theta \approx 0$ standing, growing as the thigh flexes (sitting), and approaching a right angle when the phone lies flat.

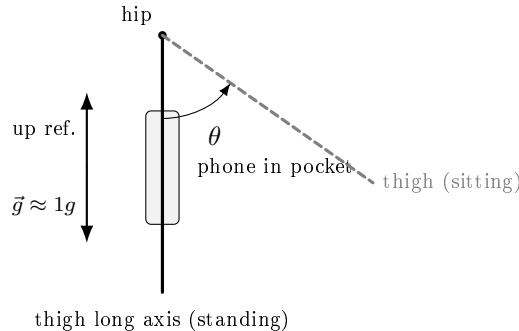


Figure 1: Phone-in-pocket geometry. Standing, gravity runs $\sim 1g$ down the thigh long axis and the calibrated “up” reference is its opposite. The run-time tilt θ is the angle between the current mean gravity vector and this standing reference; it grows as the thigh flexes (dashed, sitting).

Magnetic baseline geometry. The same standing capture also fixes the locomotion baseline. From the captured window the magnetometer yields two numbers: the field magnitude $|\vec{B}_0|$ and the inclination ι_0 , the field’s dip below horizontal measured against the gravity-derived “down” (Figure 2). `locomotion_state` stores these as the baseline and, at run time, reads `in_vehicle` when the live field has drifted past a threshold from the baseline in either magnitude or inclination — the steel mass and motor of a vehicle distort the local field. The inclination is computed gravity-only (no device quaternion), so the field’s horizontal bearing (declination) is not recovered; this is an honest limitation carried from the axis.

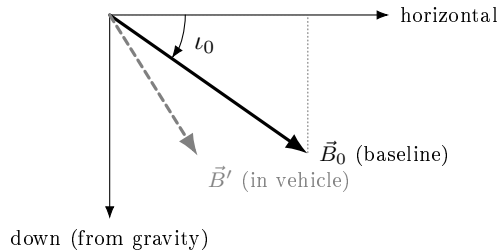


Figure 2: Magnetic baseline geometry. At calibration the standing window fixes the field magnitude $|\vec{B}_0|$ and the inclination ι_0 (its dip below horizontal, against the gravity-derived “down”). At run time `locomotion_state` reads `in_vehicle` when the live field $\vec{B}$ deviates past a threshold from this baseline in magnitude or inclination; a vehicle’s steel and motor distort the field. Inclination is gravity-only, so declination (the horizontal bearing) is not recovered.

4 The Five Axes

Each axis is summarized below: what it infers, the physical idea, the single key feature it turns on, and a pointer to its notebook. The notebooks are authoritative for the full mathematics, derivation, and validation, including reported metrics, which are deliberately not restated here. Across the axes, the rules were derived and frozen on public smartphone and wearable activity datasets — principally MotionSense [4], HHAR [8], and PAMAP2 [6] — with UCI-HAR [1] serving as the project’s baseline reference. Each axis’s specific dataset is named in its notebook.

4.1 `activity_state`

Infers how hard the body is working: sedentary, walking, running, or cycling. The physical idea is that locomotor effort shows up as the energy and cadence of body acceleration, and that magnitude features are orientation-invariant — they do not depend on how the phone sits in the pocket. The deciding features are the mean and standard deviation of the body-acceleration magnitude (in g) and the cadence from its autocorrelation; an energy split with a cadence tiebreaker resolves the walk/run overlap band, and the gyroscope is used only to separate cycling in the low-motion region. The axis reports a calibrated confidence with a reported ECE. See `maths/experiments/activity_state.ipynb` for the full method and validation.

4.2 movement_regularity

Infers how smooth and repeating the motion is, as a continuous score in $[0, 1]$ (higher is more regular). The physical idea is that regular gait makes the acceleration signal self-similar one stride later, so a short-time autocorrelation of the acceleration magnitude at the dominant lag measures regularity directly, following the trunk- accelerometry method of Moe-Nilssen and Helbostad [5]. The key quantity is the unbiased autocorrelation coefficient at the dominant period. Confidence is deterministic; there is no ECE because the score has no categorical ground-truth label to calibrate against — which is correct for a continuous score, not a gap. See `maths/experiments/movement_regularity.ipynb` for the full method.

4.3 postural_state

Infers posture relative to the body: standing, sitting, in_motion, lying, or device_resting. The physical idea is a thigh-tilt model: with the phone in the pocket, the angle of gravity relative to the calibrated standing reference reports thigh orientation. The key feature is the tilt θ , the angle between the current mean gravity vector and the calibrated standing “up” (Figure 1); standing and sitting are separable by this angle on thigh/pocket placement. The flat-orientation branch that distinguishes lying from device_resting is untested on real data and is flagged low-confidence in the reading rather than asserted. See `maths/experiments/postural_state.ipynb` for the full model.

4.4 locomotion_state

Infers whether the wearer is moving through space: stationary, on_foot, or in_vehicle. The physical idea is two-stage. Stage 1 detects on-foot travel from gait rhythm via the autocorrelation peak of body-acceleration magnitude. Stage 2, reached only when motion is low or non-rhythmic, separates stationary from in_vehicle by how far the magnetic field has drifted from its self-calibrated startup baseline (field magnitude and inclination), since a vehicle’s mass of steel and its motor distort the local field. The key feature is the baseline deviation of the field. This branch is a single-car demonstration on PAMAP2, not a validation: proper evaluation needs a transportation-mode dataset such as SHL [2], and the magnetometer baseline is sensitive to device hard-iron bias and on-body magnets. A GPS-free direction for future work is to fuse multiple vehicle cues — the $\sim 4\text{--}15$ Hz engine/road vibration band, spectral aperiodicity, and accel/brake and cornering events [3] — so that detection degrades gracefully and stays consistent with the privacy-first principle. See `maths/locomotion_state.ipynb` and `maths/experiments/locomotion_improved.ipynb` for the full method.

4.5 overall_state

Combines the four axes into a single reading: a situation label, an exertion level, and a heart-rate context note. The physical idea is that the four kinematic facts together pin down the situation more reliably than any one alone, and that a per-activity load lookup maps the situation to an expected exertion.

The combination is a small fixed pipeline (Figure 3). The situation is the *first* rule that matches in a fixed priority order; the exertion is a per-activity load lookup, forced to zero in a vehicle; and the heart-rate note follows from the exertion together with the regularity cue. The reported confidence

— the “score” — is not an average: it is the *minimum* confidence among only the axes that actually drive the chosen situation, so a combined read is only as strong as its weakest contributing ingredient.

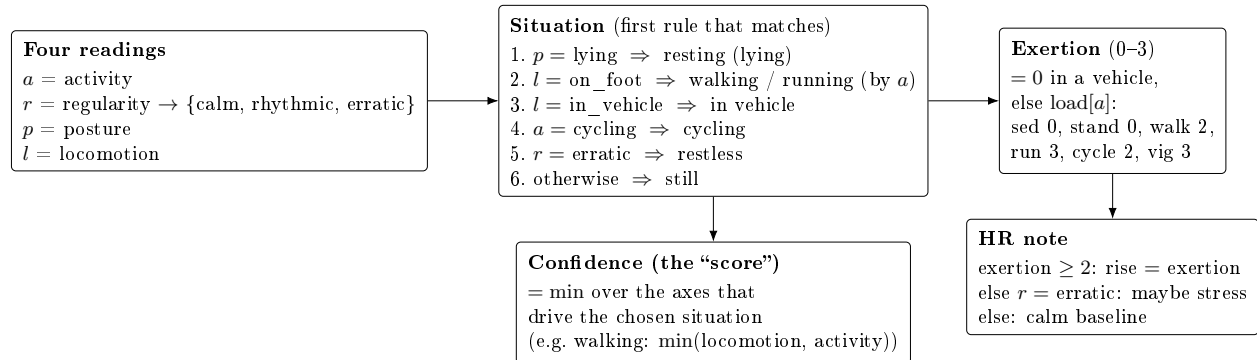


Figure 3: How `overall_state` combines the four readings. The situation is the first matching rule in a fixed priority order; the exertion is a per-activity load lookup (zeroed in a vehicle); the heart-rate note follows from the exertion and the regularity cue. The headline confidence is the minimum confidence among only the axes that drive the chosen situation.

Worked example. Take $a = \text{walking}$, regularity score 0.72 (so $r = \text{rhythmic}$), $p = \text{standing}$, and $l = \text{on_foot}$. Rule 1 does not fire ($p \neq \text{lying}$); rule 2 does ($l = \text{on_foot}$, and $a \neq \text{running}$), giving the situation **walking**. The wearer is not in a vehicle, so the exertion is $\text{load}[\text{walking}] = 2$; because $\text{exertion} \geq 2$, the note is “HR rise = physical exertion (expected)”. Walking is driven by the locomotion and activity axes, so the confidence is the smaller of those two percentages: with illustrative per-axis values of 78% (locomotion) and 84% (activity), the headline confidence is 78%, limited by locomotion. The combined reading is therefore {situation: walking, exertion: 2, note: HR rise expected, confidence: 78%}. See `maths/overall_state.ipynb` for the full combiner.

5 Convergence

The five axes were frozen separately, on different datasets and body placements, and then run together off one pocket stream. Bringing them onto a single stream is itself a test: an axis tuned on one placement or unit convention fails visibly when fed the shared pocket window, and the convergence work surfaced and fixed several such real bugs. These included unit mismatches (thresholds carried in m/s^2 applied to features expressed in g , off by a factor of ~ 9.81), a thigh-tilt inversion in which a flat reference made a flat phone read as standing, and magnetometer baselines that had to become self-calibrated at startup rather than assumed. Each fix is recorded where it was made; the convergence procedure is documented in `maths/experiments/convergence_test.ipynb`.

6 Limitations and Future Work

The honest limitations of the current system are consolidated here.

- **Placement and labeled calibration are dataset luxuries.** Several axes were derived on chest-worn or labeled data, whereas deployment is pocket-worn and unlabeled. A chest signal

is not a pocket signal, and a deployed device has no labels to calibrate against; the system therefore relies on the startup self-calibration of Section 3 to recover the references it cannot assume.

- **Locomotion is a single-car demonstration.** The stationary-versus-vehicle branch has been shown on one car in PAMAP2, not validated. Proper evaluation needs a transportation-mode dataset such as SHL [2], and the magnetometer baseline remains sensitive to device hard-iron bias and on-body magnets.
- **The lying branch is untested.** The flat-orientation distinction between lying and device_resting has no real-data validation and is reported low-confidence rather than asserted.
- **Transition windows are untested.** A 5 s window that straddles a change of activity mixes two states; behaviour across such transitions has not been characterized.
- **Thresholds are provisional.** A number of thresholds were carried from the placement or dataset they were tuned on and are marked provisional in the configuration; they need re-tuning on deployment data.
- **GPS-free multi-cue locomotion.** The most promising direction is the fused vehicle-cue detector described above [3], which would improve locomotion robustness without adding any location tracking.

References

- [1] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. In *Proceedings of the 21st European Symposium on Artificial Neural Networks (ESANN)*, 2013. UCI-HAR dataset.
- [2] Hristijan Gjoreski, Mathias Ciliberto, Lin Wang, Francisco Javier Ordóñez Morales, Sami Mekki, Stefan Valentin, and Daniel Roggen. The university of sussex-huawei locomotion and transportation dataset for multimodal analytics with mobile devices. *IEEE Access*, 6, 2018. Sussex-Huawei Locomotion (SHL) dataset.
- [3] Samuli Hemminki, Petteri Nurmi, and Sasu Tarkoma. Accelerometer-based transportation mode detection on smartphones. In *Proceedings of the 11th ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2013.
- [4] Mohammad Malekzadeh, Richard G. Clegg, Andrea Cavallaro, and Hamed Haddadi. Mobile sensor data anonymization. In *Proceedings of the International Conference on Internet of Things Design and Implementation (IoTDI)*, 2019. MotionSense dataset.
- [5] Rolf Moe-Nilssen and Jorunn L. Helbostad. Estimation of gait cycle characteristics by trunk accelerometry. *Journal of Biomechanics*, 37(1):121–126, 2004.
- [6] Attila Reiss and Didier Stricker. Introducing a new benchmarked dataset for activity monitoring. In *Proceedings of the 16th International Symposium on Wearable Computers (ISWC)*, 2012. PAMAP2 Physical Activity Monitoring dataset.
- [7] Sebastian Staacks, Simon Hütz, Heidrun Heinke, and Christoph Stampfer. Advanced tools for smartphone-based experiments: phyphox. *Physics Education*, 53(4), 2018.

- [8] Allan Stisen, Henrik Blunck, Sourav Bhattacharya, Thor Siiger Prentow, Mikkel Baun Kjærgaard, Anind Dey, Tobias Sonne, and Mads Møller Jensen. Smart devices are different: Assessing and mitigating mobile sensing heterogeneities for activity recognition. In *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2015. Heterogeneity Human Activity Recognition (HHAR) dataset.